



LI.FI

LI.FI Security Review

PaxosTransitFacet.sol(v1.0.0)

Security Researcher

Sujith Somraaj (somraajsujith@gmail.com)

Report prepared by: Sujith Somraaj

July 8, 2026

Contents

1	About Researcher	2
2	Disclaimer	2
3	Scope	2
4	Risk classification	2
4.1	Impact	2
4.2	Likelihood	3
4.3	Action required for severity levels	3
5	Executive Summary	3
6	Findings	4
6.1	Informational	4
6.1.1	Destination route is trusted from the Paxos quote but not bound to BridgeData	4
6.1.2	Swap path can pay nativeFee from pre-existing Diamond ETH	4
6.1.3	Swap output asset is not validated against the Paxos offer asset	4

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over \$170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast and Angles

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

- src/Facets/PaxosTransitFacet.sol(v1.0.0)
- src/Interfaces/IPaxosTransit.sol(v1.0.0)

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

- High** leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium** global losses <10% or losses to only a subset of users, but still unacceptable.
- Low** losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

- High** almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium** only conditionally possible or incentivized, but still relatively likely
- Low** requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

- Critical** Must fix as soon as possible (if already deployed)
- High** Must fix (before deployment if not already deployed)
- Medium** Should fix
- Low** Could fix

5 Executive Summary

Over the course of 7 hours in total, [LI.FI](#) engaged with the [researcher](#) to audit the contracts described in section 3 of this document ("scope").

In this period of time a total of 3 issues were found.

Project Summary	
Project Name	LI.FI
Repository	lifinance/contracts
Commit	149e073de
Audit Timeline	July 08, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found	
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	0
Gas Optimizations	0
Informational	3
Total Issues	3

6 Findings

6.1 Informational

6.1.1 Destination route is trusted from the Paxos quote but not bound to BridgeData

Context: [PaxosTransitFacet.sol#L169](#)

Description: `_startBridge` validates source asset, receiver, and distributor code, but intentionally does not validate `quote.route.destEID` or `quote.route.wantAsset` against `_bridgeData.destinationChainId` or expected destination asset.

Funds follow the Paxos-signed quote, while LI.FI events and clear- signing metadata reflect BridgeData. If these values diverge, users, indexers, support tooling, and monitoring systems can observe one destination in LI.FI data while the Paxos order routes funds to another chain or asset.

Recommendation: Document this trust assumption clearly for off-chain actors. Indexers, clear-signing surfaces, support tooling, and monitoring systems should treat the Paxos-signed quote as the source of truth for destination routing and destination asset, not `_bridgeData.destinationChainId` or other LI.FI event metadata.

LI.FI: Acknowledged — this is intentional. Funds always follow the Paxos-signed, backend-generated quote (same trust model as `AcrossFacetV4`'s `outputAmount`), so `destEID/wantAsset` are the source of truth while `_bridgeData.destinationChainId` is analytics/events-only. The trust assumption is already documented in the code NOTE and in `docs/PaxosTransitFacet.md` ("Not enforced on-chain"), and we'll ensure off-chain indexers/monitoring treat the quote as authoritative for destination routing.

Researcher: Acknowledged.

6.1.2 Swap path can pay `nativeFee` from pre-existing Diamond ETH

Context: [PaxosTransitFacet.sol#L131](#)

Description: The non-swap path prevents fee payment from Diamond balance by requiring `nativeFee <= msg.value`.

The swap path skips this so ERC20-to-native swaps can fund the fee, but it does not verify that the fee was actually funded by `msg.value` or swap output. `submitOrder{ value: nativeFee }` can therefore consume ETH already sitting in the Diamond.

Recommendation: Snapshot initial native balance excluding `msg.value`, then after swaps require `address(this).balance >= initialNativeBalance + nativeFee` before calling Paxos.

LI.FI: Acknowledged. Impact is bounded to any incidental native balance in the Diamond, which is not designed to custody funds, and this matches the existing pattern in `StargateFacetV2/GlacierFacet` (which pass `nativeFee` as `nativeReserve` without a funding check). We treat this as accepted risk given the Diamond holds no persistent balance, and only ever route through backend-generated calldata.

Researcher: Acknowledged.

6.1.3 Swap output asset is not validated against the Paxos offer asset

Context: [PaxosTransitFacet.sol#L101](#)

Description: In `swapAndStartBridgeTokensViaPaxosTransit`, `_depositAndSwap` measures `receivedAmount` from the last swap's `receivingAssetId`, but `_startBridge` later approves and submits `_bridgeData.sendingAssetId`, which is only checked against `quote.route.offerAsset`.

There is no validation that the final swap output is the same token being bridged. This can make slippage checks and positive-slippage refunds operate on the wrong asset, and can lock swap output or consume pre-existing Diamond balances if present.

Recommendation: Before `_depositAndSwap`, require the last swap's `receivingAssetId` to equal `_bridgeData.sendingAssetId`, reverting with `InformationMismatch`.

LI.FI: Fixed in [50581ab](#)

Researcher: Verified fix.